

# AEGISPAY

## Agentic Commerce Architecture V4

*Different protocols can enter AegisPay,  
but none can bypass the AegisPay Control Plane.*

Grow | Sell | Protect | Protocol Gateway  
FastAPI | LangGraph | PostgreSQL | Redis | AWS | Razorpay

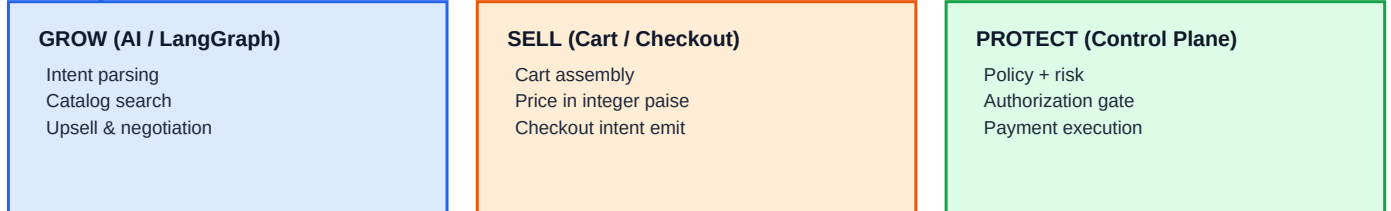
*August 2026*

## 1. Engineering Philosophy

AI models are non-deterministic. They are good at language, negotiation, and routing. They are not trusted to move money. AegisPay keeps a strict air-gap: agents and external protocols may propose commerce actions, but only the deterministic Control Plane may authorize and execute payment.

V4 adds a Protocol Gateway so UCP, ACP, AP2, A2A, MCP, x402, A2UI, and India rails (NPCI UAP / UPI) can enter through adapters - not through separate payment stacks. Every adapter normalizes to one AegisPay Intent. One policy engine. One audit trail.

### Air-Gap Model: Grow / Sell / Protect

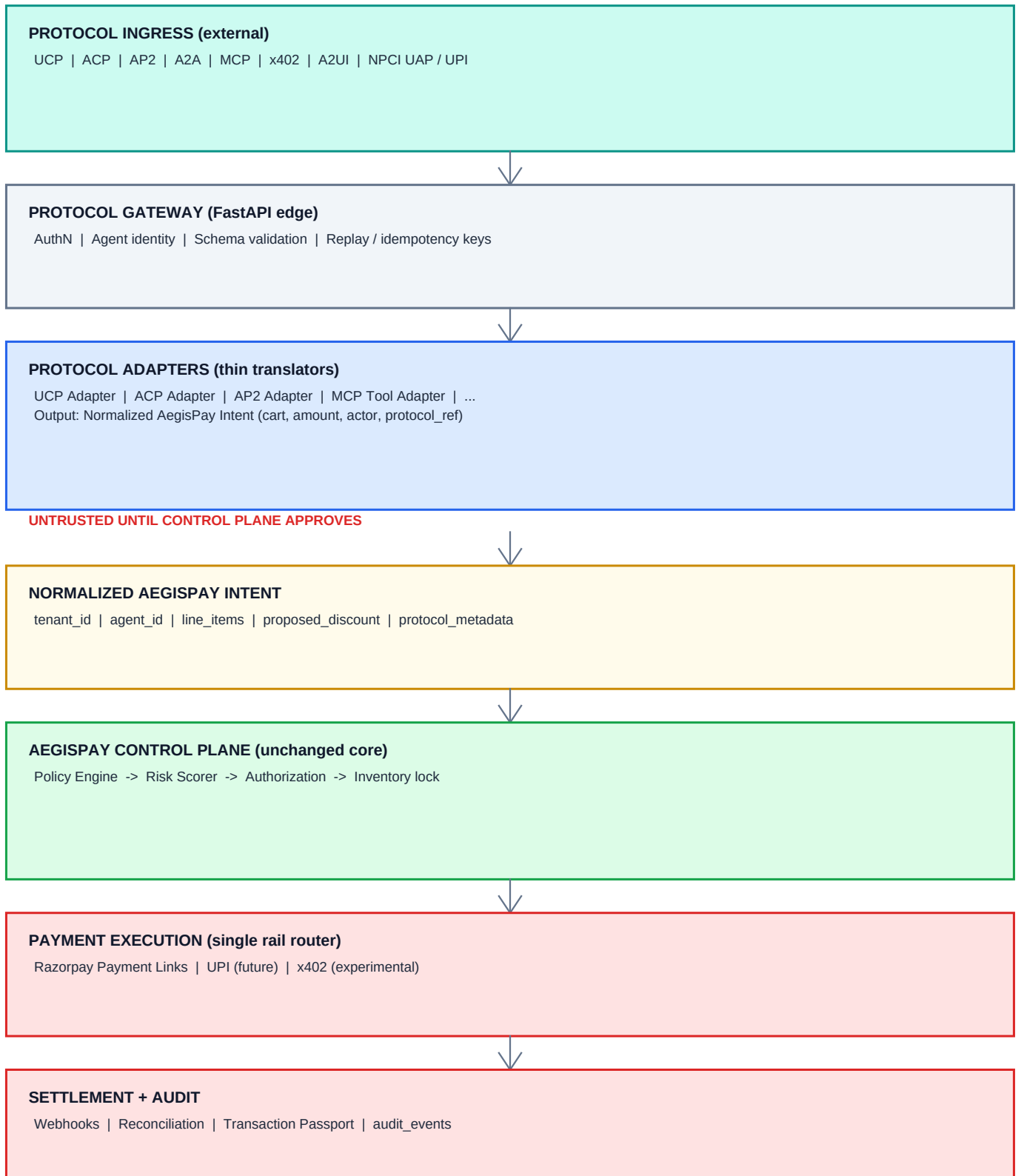


### Trust Boundaries

- UNTRUSTED: protocol payloads, user input, LLM output, inbound webhooks
- SEMI-TRUSTED: LangGraph + Protocol Adapters (propose only)
- TRUSTED: Control Plane, integer math, scoped authorization, payment clients

## 2. Protocol Gateway (V4)

The Protocol Gateway is the single front door for agentic-commerce protocols. It authenticates the caller, validates schema, maps protocol-specific messages to a Normalized AegisPay Intent, and forwards that intent to the existing Control Plane. No adapter calls Razorpay, UPI, or x402 directly.



### 3. Protocol Catalog & Maturity

Status reflects architecture support today - not marketing claims. Core = in the live path. Adapter-ready = gateway contract defined, adapter can ship. Experimental = stub only. Future = planned rail, no adapter code yet.

| Protocol | Role                                | Status        | AegisPay mapping                                                       |
|----------|-------------------------------------|---------------|------------------------------------------------------------------------|
| MCP      | Controlled agent tools & context    | Core          | Tool allowlist on agents table; MCP server maps to approved tools only |
| ACP      | Agentic checkout / commerce         | Core          | Same as SELL path: cart -> guardrail -> pay                            |
| A2UI     | Agent-driven UI components          | Adapter-ready | Split UI: chat + trace; A2UI renders into hosted surfaces              |
| UCP      | Commerce interoperability           | Adapter-ready | Adapter normalizes catalog/cart to AegisPay Intent                     |
| AP2      | Payment mandates & verifiable auth  | Adapter-ready | Maps to authorizations table (scoped, expiring, single-use)            |
| A2A      | Agent-to-agent messaging            | Experimental  | Gateway accepts A2A envelope; no production adapter yet                |
| x402     | Machine / pay-per-use HTTP payments | Experimental  | Rail router stub; does not bypass Control Plane                        |
| NPCI UAP | India unified agentic payments      | Future        | Architecture slot in rail router; no live integration                  |
| UPI      | India instant payments              | Future        | Settlement rail behind same authorization object                       |

#### What we do NOT do

- Separate payment systems per protocol
- LLM-direct Razorpay / UPI / x402 calls
- Protocol-specific policy engines
- Claiming live UPI/x402 until adapter + rail tests pass

## 4. System Architecture

Client and protocol traffic enters through the API Gateway. LangGraph runs GROW and SELL. The Control Plane is protocol-agnostic. PostgreSQL + RLS hold catalog, orders, and audit. Redis handles locks, rate limits, and trace pub/sub.



## 5. Workflow: GROW, SELL, PROTECT

Lifecycle stays linear. Protocol origin does not shorten the path.

### Phase 1 - GROW

#### 1. Intent

Buyer or external agent requests a product, bundle, or discount.

#### 2. Catalog

Postgres search - not LLM hallucination.

#### 3. Upsell

Agent proposes bundle or discount percentage (not final price).

#### 4. Intent emit

Adapter or LangGraph emits Normalized AegisPay Intent.

### Phase 2 - PROTECT

#### 1. Inventory lock

Redis + Postgres hold; fail closed if stock unavailable.

#### 2. Policy

Max discount, margin floor, qty caps - Python only.

#### 3. Risk

Injection patterns, velocity, anomaly score.

#### 4. Authorization

Scoped permission bound to cart hash + amount; single-use.

### Phase 3 - SELL

#### 1. User confirm

Explicit confirm - LangGraph interrupt; not inferred from chat.

#### 2. Idempotent order

Idempotency key per offer version; cart hash bound.

#### 3. Payment link

Razorpay POST /v1/payment\_links (test or live).

#### 4. Webhook

HMAC verify, dedupe event\_id, amount match, mark paid.

#### 5. Passport

audit\_events + order snapshot for replay.

Protocol -> Gateway -> Intent -> GROW/SELL -> PROTECT -> [APPROVED] -> Pay -> Settle

## 6. Payment State Machine

Same state machine for all rails. UNKNOWN is a real state - never blind retry.

| State           | Next                 | Notes                        |
|-----------------|----------------------|------------------------------|
| created         | payment_pending      | Order + authorization issued |
| payment_pending | captured             | Webhook or poll confirms pay |
| captured        | completed            | Reconciliation OK            |
| payment_pending | unknown              | Timeout / no webhook         |
| unknown         | reconciliation       | Ask provider truth           |
| unknown         | captured / failed    | After reconcile              |
| authorization   | approved -> consumed | Single-use scope             |
| authorization   | expired / revoked    | No pay allowed               |

### Webhook + reconciliation (all rails)

- Verify signature (HMAC for Razorpay)
- Dedupe by provider\_event\_id
- Amount must match authorized amount\_minor
- Mismatch -> disputed state, HITL P0, no auto-complete
- Outbox worker publishes events; reconciliation worker resolves UNKNOWN

## 7. Protocol Gateway Security

Every protocol adapter runs the same security checks before the Control Plane sees data.

| Control              | Gateway enforcement                                             |
|----------------------|-----------------------------------------------------------------|
| Authentication       | mTLS or signed JWT per protocol; no anonymous money ingress     |
| Agent identity       | agents table: type, scopes, allowed_tools, trust_level, expiry  |
| Schema validation    | Pydantic / JSON Schema per adapter; reject malformed payloads   |
| Replay protection    | nonce + timestamp window; idempotency_keys per scope            |
| Scoped authorization | AP2 mandates map to authorizations; bound to cart_hash + amount |
| Idempotency          | Same key returns same result; no double spend                   |
| Tool allowlisting    | MCP tools must appear in agents.allowed_tools JSONB             |
| Tenant isolation     | tenant_id on every row; Postgres RLS; SET LOCAL per request     |



## 8. Hardcoded Guardrails

No protocol or LLM prompt overrides these. Enforced in Control Plane code.

| Vector            | Threat                      | AegisPay response                        |
|-------------------|-----------------------------|------------------------------------------|
| Cross-tenant leak | Protocol sends wrong tenant | RLS + server-set tenant context          |
| Double spend      | Retry after timeout         | Idempotency + never-retry-unknown        |
| Margin erosion    | Injection / fake discount   | RULE_MIN_MARGIN + price floor            |
| Oversell          | Two agents, one SKU         | Redis lock + FOR UPDATE holds            |
| Untraceable pay   | Dispute with no proof       | Transaction Passport from audit_events   |
| Forged webhook    | Fake paid event             | HMAC + event dedupe + amount check       |
| LLM math          | Wrong total                 | Integer paise in compute_totals only     |
| Protocol bypass   | Adapter calls Razorpay      | Adapters cannot hold payment credentials |

### Authorization outcomes

|                     |                                     |
|---------------------|-------------------------------------|
| <b>AUTO-APPROVE</b> | Low risk, in policy, stock OK       |
| <b>CLAMP</b>        | Discount over cap - reduce to max   |
| <b>REJECT</b>       | Margin or stock fail                |
| <b>HUMAN</b>        | High risk, max rounds, engine error |

## 9. Transaction Passport & Audit

No separate passport table. Built from order, authorization, policy version, payment, and hash-chained audit\_events. Append-only. Tamper-evident.

### Passport sources

|                               |                                                |
|-------------------------------|------------------------------------------------|
| <code>protocol_ref</code>     | Which adapter / protocol originated the intent |
| <code>session_id</code>       | Agent session or LangGraph thread              |
| <code>authorization_id</code> | Scoped approval used for payment               |
| <code>policy_version</code>   | Ruleset that ran                               |
| <code>grow_trace</code>       | Intent, catalog, negotiation                   |
| <code>protect_trace</code>    | Per-rule pass / clamp / reject                 |
| <code>sell_trace</code>       | Rail, idempotency_key, webhook ids             |
| <code>amount_minor</code>     | Final integer amount                           |

### Observability

- WebSocket trace:{session\_id} - live node + rule events
- audit\_events - durable ledger
- Redis pub/sub - fan-out to trace UI

## 10. AWS Deployment

Monolithic API first. Workers for webhooks and reconciliation. No per-protocol stacks.

| Layer   | AWS service       | Purpose                                |
|---------|-------------------|----------------------------------------|
| Edge    | ALB + WAF         | TLS termination, rate limits           |
| Compute | ECS Fargate / EKS | FastAPI + LangGraph + Protocol Gateway |
| Data    | RDS PostgreSQL    | RLS catalog, orders, audit_events      |
| Cache   | ElastiCache Redis | Locks, sessions, trace pub/sub         |
| Secrets | Secrets Manager   | Razorpay keys - never in DB or LLM     |
| Async   | SQS + workers     | webhook_events, outbox, reconciliation |
| Static  | S3 + CloudFront   | A2UI assets, exports                   |
| Logs    | CloudWatch        | Structured audit correlation_id        |

### Human-in-the-loop

- escalation\_tickets for ESCALATED guardrail outcomes
- Human override logged with actor=human; margin floor not overridden in v1
- Payment disputed (webhook mismatch) -> P0 queue

## **11. V4 Summary**

AegisPay V4 welcomes the agentic-commerce protocol ecosystem through one Protocol Gateway and one Control Plane. GROW and SELL are unchanged in spirit. PROTECT is unchanged in law: policy, risk, and authorization run in deterministic Python before any rail is called.

### **Implementation truth (Aug 2026)**

- Core today: MCP tool gating, ACP/SELL checkout, Razorpay, audit, RLS, HITL design
- Adapter-ready: UCP, AP2, A2UI gateway contracts documented
- Experimental: A2A ingress envelope, x402 rail stub
- Future: NPCI UAP / UPI settlement adapters